

Zynq UltraScale+ RFSoc DFE를 위한 2계층 A/B 이중화 부팅 및 OTA 업데이트 아키텍처 구현 백서 (PoC 가이드)

개요: Executive Summary

본 보고서는 AMD Zynq UltraScale+ RFSoc DFE (Digital Front-End)¹ 플랫폼에서 최고 수준의 시스템 안정성 및 현장 펌웨어 업데이트(Over-the-Air, OTA) 기능을 구현하기 위한 A/B 이중화 부팅 아키텍처의 설계 및 단계별 구현 가이드를 제공합니다. RFSoc DFE 디바이스는 Zynq MPSoC 아키텍처를 기반으로 하며³, 본질적으로 복잡한 다단계 부트 시퀀스를 공유합니다.⁴ 시스템 장애 또는 업데이트 실패 시 '브릭(brick)' 상태를 방지하기 위해, 본 문서는 FSBL(First Stage Bootloader)부터 U-Boot, Linux Kernel 및 Root File System에 이르는 전체 부트 체인을 이중화하는 '2계층(Two-Tiered)' 이중화 전략을 제시합니다.

1계층(Tier 1)은 Zynq MPSoC의 하드웨어 BootROM 기능을 활용하여 BOOT.BIN 자체를 이중화합니다. 이는 CSU_MULTI_BOOT 레지스터⁶를 통해 제어되며, 부트로더의 치명적인 손상 시에도 하드웨어 레벨의 'Golden Image' 복구를 보장합니다.⁸

2계층(Tier 2)은 U-Boot 부트로더 내부의 스크립트 로직을 통해 Linux Kernel 및 Root File System의 A/B 파티션을 선택합니다. U-Boot의 bootcount 메커니즘⁹과 RAUC¹⁰ 및 Mender¹² 같은 프레임워크에서 사용하는 시도 횟수 기반 로직¹¹을 결합하여, 부팅 실패 시 자동으로 정상 파티션으로 페일오버합니다.

본 보고서는 PetaLinux 개발 환경을 사용하여 PoC(Proof of Concept) 수준에서 이 아키텍처를 구축하는 전 과정을 상세히 기술합니다. 여기에는 플래시 파티션 설계, PetaLinux MTD 파티션 및 디바이스 트리 구성¹⁵, bootgen을 이용한 이중 BOOT.BIN 생성¹⁸, 커스텀 U-Boot boot.scr A/B 페일오버 스크립트 작성¹⁴, fw_setenv를 통한 Linux 유저스페이스 연동²¹, 그리고 Cadence 하드웨어 워치독(WDT)을 이용한 시스템 행(hang) 감지²³가 포함됩니다.

독자는 이 PoC 가이드를 통해 Zynq RFSoc 플랫폼에서 즉시 적용 가능한 강력한 이중화 부팅 시스템을 구현하고, 이를 기반으로 프로덕션 레벨의 OTA 업데이트 솔루션으로 확장할 수 있는 핵심 기술을 습득하게 될 것입니다.

섹션 1: Zynq RFSoc를 위한 A/B 이중화 부팅 아키텍처 설계

(Architecture Design for A/B Redundancy on Zynq RFSoc)

Zynq UltraScale+ RFSoc 디바이스의 부팅 프로세스는 복잡한 다단계 시퀀스를 따릅니다.³ 사용자의 요구사항(FSBL부터 커널까지의 이중화)을 만족시키기 위해서는 단일 A/B 파티션 전략이 아닌, 하드웨어와 소프트웨어가 결합된 2계층 이중화 아키텍처가 필수적입니다.

1.1 Zynq MPSoC/RFSoc 표준 부트 플로우 분석

1. **CSU BootROM:** 전원이 인가되면, PS(Processing System)의 CSU(Configuration Security Unit)에 내장된 BootROM이 실행됩니다. BootROM은 부트 모드 핀⁴을 읽어 QSPI, SD, eMMC 등 부트 디바이스를 결정합니다.³
2. **Golden Image Search:** BootROM은 QSPI 플래시의 0x0 주소부터 시작하여 32KB²⁵ 간격으로 유효한 부트 헤더(XLNX 시그니처)를 검색합니다.⁶
3. **FSBL 로드:** 유효한 헤더를 찾으면, BootROM은 해당 BOOT.BIN의 FSBL(First Stage Bootloader) 파티션을 OCM(On-Chip Memory)으로 로드하고 실행을 넘깁니다.²⁷
4. **FSBL 실행:** FSBL은 BOOT.BIN에 포함된 나머지 파티션들을 로드합니다. 일반적인 PetaLinux 부팅 순서는 다음과 같습니다²⁷:
 1. PMU 펌웨어 (PMUFW) 로드²⁹
 2. (선택적) PL Bitstream 로드
 3. ARM Trusted Firmware (ATF, bl31.elf) 로드²⁷
 4. 2단계 부트로더 (U-Boot, u-boot.elf) 로드 및 실행
5. **U-Boot 및 Linux 부팅:** U-Boot가 실행되면, boot.scr 스크립트를 로드하여²⁰ Linux 커널(image.ub)을 메모리에 로드하고, 커널 bootargs를 설정한 뒤¹⁴ 커널에 제어권을 넘깁니다.

1.2 2계층 이중화 전략: 하드웨어 및 소프트웨어 페일오버의 결합

단순히 Linux RootFS만 A/B로 나누는 것은 FSBL이나 U-Boot가 손상되었을 때 시스템을 복구할 수 없습니다. 사용자의 요청을 완벽히 충족하기 위해 두 개의 독립적인 페일오버 계층을

설계합니다.

- 계층 1: **BOOT.BIN** 이중화 (하드웨어 레벨 파일오버)
 - 목적: FSBL, ATF, U-Boot를 포함하는 전체 부트로더 체인의 손상에 대비합니다.
 - 구현: Zynq MPSoC의 하드웨어 MultiBoot 기능을 활용합니다.⁶
 - QSPI 플래시 저장:
 - BOOT_A ("Golden"): 0x0 주소에 위치. 검증된 안정 버전.
 - BOOT_B ("Update"): 0x<MultiBoot_Offset> (예: 0x20000000)에 위치. 업데이트용.
 - 동작: BootROM은 CSU_MULTI_BOOT 레지스터⁷ 값을 읽어 부트 오프셋을 결정합니다. 기본값(0)은 0x0 (BOOT_A)에서 부팅을 시도합니다.
- 계층 2: **Kernel/RootFS** 이중화 (소프트웨어 레벨 파일오버)
 - 목적: Linux 커널, 디바이스 트리 또는 RootFS의 손상, 부팅 실패, 런타임 행(hang)에 대비합니다.
 - 구현: 계층 1에서 성공적으로 로드된 U-Boot 내부의 스크립트 로직을 활용합니다.³⁵
 - QSPI 플래시 저장:
 - System_A (kernel_a, rootfs_a): BOOT_A와 연동되는 Golden 시스템.
 - System_B (kernel_b, rootfs_b): BOOT_B와 연동되는 Update 시스템.

1.3 핵심 아키텍처: 2계층 상호작용 및 신뢰 체인

본 PoC 아키텍처는 단순한 이중화가 아닌, 각 계층의 장애 감지가 연쇄적으로 작동하여 '절대 실패하지 않는(never-brick)' 시스템을 구축하는 데 목적이 있습니다.

1.3.1 2계층 이중화 아키텍처의 필요성

사용자는 FSBL부터 커널까지의 이중화를 요청했습니다. 이를 위해 Zynq BootROM이 CSU_MULTI_BOOT 레지스터를 통해 BOOT.BIN 이미지 자체의 오프셋을 선택하는 하드웨어 MultiBoot 기능⁶과, U-Boot가 boot.scr과 환경 변수를 통해 커널/RootFS A/B 슬롯을 선택하는 소프트웨어 로직¹¹을 결합해야 합니다.

- 1계층 (Tier 1 - 하드웨어): BOOT.BIN 선택. (BootROM 담당)
- 2계층 (Tier 2 - 소프트웨어): Kernel/RootFS 선택. (U-Boot 담당)

본 PoC에서는 두 개의 BOOT.BIN (A/B)이 동일한 U-Boot 로직을 공유하고, 이 U-Boot가 Kernel/RootFS (A/B)를 선택하도록 설계합니다. 이는 BOOT.BIN의 손상(1계층)과 커널의 손상(2계층)을 모두 처리할 수 있는 유연한 구조입니다.

1.3.2 부트 실패 감지의 연쇄 작용 (Chain of Failure Detection)

장애 상황은 여러 지점에서 발생하며, 각기 다른 메커니즘으로 감지되어 다음 단계의 페일오버를 트리거합니다.

1. **(Linux 행 감지):** Linux 런타임 중 시스템이 멈추면 ²³, 하드웨어 워치독(WDT)이 타임아웃되고 PMUFW가 시스템 리셋을 요청하여 ²⁴ 시스템이 재부팅됩니다.
2. **(U-Boot 계층 2 페일오버):** 시스템이 재부팅되면 U-Boot가 실행되고, bootcount 또는 boot_x_left 변수를 증가/감소시킵니다.⁹ U-Boot는 임계값에 도달하면 대체 커널 슬롯(Tier 2)으로 부팅을 시도합니다.
3. **(BOOT.BIN 손상 감지):** BOOT.BIN 헤더가 손상되면 ⁸, BootROM이 헤더 검증에 실패하고 자동으로 32KB 오프셋을 검색하여 ⁶ 대체 BOOT.BIN 슬롯(Tier 1)을 찾아 부팅합니다.

가장 치명적인 시나리오는 BOOT.BIN의 헤더는 유효하지만 FSBL이나 U-Boot 코드가 손상되어 부팅 중 멈추는 경우입니다.³⁸ BootROM은 이를 감지하지 못하고 제어권을 넘기며, 시스템은 '브릭' 상태가 됩니다.

본 PoC는 이 시나리오를 해결하기 위해 (1)의 WDT 메커니즘을 (2)의 U-Boot bootcount와 연동합니다. Linux 부팅이 성공적으로 완료되면(즉, systemd가 워치독을 'pet'하기 시작하면), 유저스페이스 스크립트가 fw_setenv를 실행하여 U-Boot의 bootcount를 0으로 리셋합니다.⁹ 만약 Linux가 WDT를 'pet'하지 못할 정도로 심각하게 실패하면, WDT가 시스템을 리셋하고 U-Boot는 bootcount가 리셋되지 않았음을 확인하고 증가시킵니다.

이 bootcount가 bootlimit에 도달하면 ⁹, U-Boot는 altbootcmd를 실행하여 다른 BOOT.BIN 슬롯(Tier 1)으로 부팅하도록 CSU_MULTI_BOOT 레지스터를 직접 조작 ⁴²한 후 리셋합니다. 이는 소프트웨어(Tier 2) 장애가 하드웨어(Tier 1) 페일오버를 트리거하는 강력한 신뢰 체인입니다.

섹션 2: PetaLinux 프로젝트 설정 및 A/B 파티션 스키마 정의

(PetaLinux Project Setup and A/B Partition Schema Definition)

PoC 구현의 첫 번째 단계는 PetaLinux 프로젝트를 생성하고 A/B 아키텍처의 기반이 되는 플래시 파티션 레이아웃을 정의하는 것입니다. 본 가이드는 128MB (1024Mbit) QSPI 플래시를 기준으로 합니다.

2.1 PetaLinux 프로젝트 생성 (Zynq RFSoc DFE 기준)

1. **Vivado** 하드웨어 설계 (**XSA**): RFSoc DFE 디자인¹을 포함하는 Vivado 프로젝트를 빌드하고 하드웨어 정의 파일(.xsa)을 Export합니다. 이 단계에서 QSPI 인터페이스와 Cadence WDT(FPD WDT)가 PS에서 활성화되어야 합니다.²³
2. **PetaLinux** 환경 설정: `source <petalinux_install_dir>/settings.sh`
3. **PetaLinux** 프로젝트 생성:⁴³
Bash
`petalinux-create --type project --template zynqMP --name rfsoc_ab_boot`
`cd rfsoc_ab_boot`
`petalinux-config --get-hw-description=<path_to_xsa_file>`
4. `petalinux-config` 메뉴가 나타나면, 부트 디바이스 설정을 확인합니다.
 - Subsystem AUTO Hardware Settings ---> Advanced bootable images storage Settings ---> boot image settings ---> image storage media를 primary flash로 설정합니다.
 - Subsystem AUTO Hardware Settings ---> Advanced bootable images storage Settings ---> kernel image settings ---> image storage media를 primary flash로 설정합니다.⁴⁶

2.2 A/B QSPI 플래시 파티션 레이아웃 설계 (PoC)

신뢰성 있는 A/B 시스템은 부트로더 환경 변수(`bootenv`)와 사용자 데이터를 위한 별도 파티션을 포함해야 합니다.⁴⁷ U-Boot 환경 변수(`boot_order`, `boot_a_left` 등)는 A/B 상태를 관리하는 핵심입니다.⁴⁸ `saveenv` 명령은 이 변수들을 `bootenv` 파티션에 씁니다.⁴⁹

만약 `bootenv` 파티션이 하나만 있고, `saveenv` (플래시 쓰기) 도중 전원이 차단되면, 전체 A/B 상태 정보가 손상되어 시스템이 부팅 불능 상태에 빠질 수 있습니다. U-Boot는 `CONFIG_ENV_OFFSET_REDUND` 옵션을 통해 이중 환경 변수 저장을 지원합니다.²¹ 따라서 `bootenv`와 `bootenv_redund` 두 개의 파티션을 생성하여, U-Boot가 자동으로 이 두 파티션에 번갈아 쓰도록 설정합니다.⁵¹

표 1: A/B QSPI 플래시 파티션 맵 (PoC - 128MB QSPI 기준)

- 배경: 이 테이블은 PoC를 위한 구체적인 메모리 맵입니다. Zynq MPSoC의 MultiBoot 오프셋은 32KB 정렬이 필수입니다.⁶ 128MB QSPI (0x8000000)를 기준으로, `boot_a`

(Golden)는 0x0에 ⁶, boot_b (Update)는 0x2000000 (32MB)에 배치합니다.⁸

파티션 이름 (Label)	시작 오프셋	크기	MTD 이름 (예상)	용도 및 설명
boot_a	0x00000000 0	32M	mtd0	Golden BOOT.BIN (FSBL, PMUFW, ATF, U-Boot)
boot_b	0x02000000 0	32M	mtd1	Update BOOT.BIN (FSBL, PMUFW, ATF, U-Boot)
bootenv	0x04000000 0	256K	mtd2	U-Boot 환경 변수 (A/B 상태 저장)
bootenv_redund	0x04040000	256K	mtd3	U-Boot 이중화 환경 변수
kernel_a	0x04080000	28M	mtd4	Golden Linux Kernel (image.ub)
rootfs_a	0x05C80000	19M	mtd5	Golden Root File System (JFFS2/UBIFS)
kernel_b	0x07000000	28M	mtd6	Update Linux Kernel (image.ub)
rootfs_b	0x07C00000 0	19M	mtd7	Update Root File System (JFFS2/UBIFS)
user_data	0x07F80000	0.5M	mtd8	사용자 데이터 (업데이트 후에도 유지)

(참고: 총 128MB. 오프셋과 크기는 예시이며, 실제 바이너리 크기에 맞게 조정되어야 합니다.)

2.3 petalinux-config 파티션 설정과 디바이스 트리의 관계

petalinux-config의 플래시 설정¹⁵은 주로 PetaLinux의 petalinux-package와 petalinux-boot --jtag 명령이 이미지를 플래시에 배치할 위치를 결정하는 데 사용됩니다.⁵³

하지만, Linux 커널이 부팅된 후 cat /proc/mtd로 보는 MTD 파티션⁵⁶은 이 설정이 아니라 디바이스 트리(DTB)에 의해 결정됩니다. 이 둘이 일치하지 않으면 U-Boot는 kernel_a 파티션에 쓰지만, Linux는 mtd4를 다른 주소로 인식하여 부팅에 실패하는 심각한 불일치가 발생할 수 있습니다.⁵⁸

따라서 petalinux-config 설정은 기본값으로 두거나 최소한으로 설정하고, A/B 파티션의 "진짜" 정의는 system-user.dtsi에서 수동으로 오버라이드해야 합니다.¹⁷

2.4 디바이스 트리(system-user.dtsi) A/B MTD 파티션 맵 구현

PetaLinux 프로젝트의 project-spec/meta-user/recipes-bsp/device-tree/files/system-user.dtsi 파일을 엽니다.¹⁵ 표 1의 레이아웃을 반영하여 QSPI 플래시 노드(&qspi)를 오버라이드합니다.

DTS

```
/include/ "system-conf.dtsi"
/{
};

/*
 * QSPI A/B Partition Layout Override [17]
 * This ensures the Linux Kernel MTD driver [57] matches our
 * intended layout from Table 1.
 */
&qspi {
    status = "okay";
    flash0: flash@0 {
        compatible = "jedec,spi-nor"; /* or "micron,n25q128a13" etc [58, 62] */
        reg = <0x0>;
    }
}
```



```
#address-cells = ;
```

```
#size-cells = ;
```

```
/* Delete auto-generated partitions if any [59, 63] */
```

```
/delete-node/ partition@0;
```

```
/delete-node/ partition@1;
```

```
/* Add our custom A/B partition map based on Table 1 */
```

```
partition@0 {
```

```
    label = "boot_a";
```

```
    reg = <0x00000000 0x02000000>; /* 32M */
```

```
};
```

```
partition@2000000 {
```

```
    label = "boot_b";
```

```
    reg = <0x02000000 0x02000000>; /* 32M */
```

```
};
```

```
partition@4000000 {
```

```
    label = "bootenv";
```

```
    reg = <0x04000000 0x00040000>; /* 256K */
```

```
};
```

```
partition@4040000 {
```

```
    label = "bootenv_redund";
```

```
    reg = <0x04040000 0x00040000>; /* 256K */
```

```
};
```

```
partition@4080000 {
```

```
    label = "kernel_a";
```

```
    reg = <0x04080000 0x01C00000>; /* 28M */
```

```
};
```

```
partition@5C80000 {
```

```
    label = "rootfs_a";
```

```
    reg = <0x05C80000 0x01380000>; /* 19M */
```

```
};
```

```
partition@7000000 {
```

```
    label = "kernel_b";
```

```
    reg = <0x07000000 0x01C00000>; /* 28M */
```

```
};
```

```
partition@7C00000 {
```

```
    label = "rootfs_b";
```

```
    reg = <0x07C00000 0x00380000>; /* 3.5M */
```

```
};
```

```
partition@7F80000 {
```

```
    label = "user_data";
```

```
    reg = <0x07F80000 0x00080000>; /* 0.5M */
```

```
};  
};  
};
```

섹션 3: 하드웨어 레벨 **BOOT.BIN** 이중화 구현 (Tier 1)

(Implementing Hardware-Level BOOT.BIN Redundancy - Tier 1)

이 섹션에서는 Vitis/Bootgen을 사용하여 QSPI 플래시의 서로 다른 오프셋에 위치할 두 개의 BOOT.BIN (Golden/Update) 이미지를 생성합니다.

3.1 BOOT.BIN 생성을 위한 PetaLinux 빌드 아티팩트 준비

1. PetaLinux 프로젝트를 빌드하여 BOOT.BIN의 구성 요소를 생성합니다.⁴⁴

```
Bash  
petalinux-build
```

2. 빌드된 이미지는 images/linux/ 디렉토리에 위치합니다: zynqmp_fsbl.elf, pmufw.elf, bl31.elf, u-boot.elf.

3.2 이중 BOOT.BIN 생성을 위한 BIF(Boot Image Format) 파일 작성

Bootgen 유틸리티는 BIF 파일(.bif)을 입력으로 받아 BOOT.BIN을 생성합니다.³⁰ A/B 이미지를 위해 두 개의 BOOT.BIN을 생성해야 합니다.

MultiBoot 오프셋의 올바른 이해: Bootgen BIF 파일의 [offset] 속성⁶⁹은 BOOT.BIN 파일 내의 개별 파티션(예: u-boot.elf)이 시작되는 상대 주소를 지정합니다. Zynq MPSoC의 하드웨어 MultiBoot⁶는 BOOT.BIN 파일 자체가 플래시의 32KB 경계 오프셋에 위치하는 것을 감지합니다.⁶

따라서, BOOT_A.bin과 BOOT_B.bin을 생성하기 위해 bootgen을 두 번 실행합니다. BIF 파일은 동일할 수 있으며⁶, 이 두 개의 별도 파일을 플래시의 다른 주소 (예: 0x0과 0x2000000)에 프로그래밍하는 것이 올바른 절차입니다.

- **ab_boot.bif** (공통 BIF 파일): ¹⁸

코드 스니펫

```
//arch = zynqmp; split = false; format = BIN
the_ROM_image:
{
  [bootloader, destination_cpu=a53-0] images/linux/zynqmp_fsbl.elf
  [pmufw_image] images/linux/pmufw.elf
  [destination_cpu=a53-0, exception_level=el-3] images/linux/bl31.elf
  [destination_cpu=a53-0, exception_level=el-2] images/linux/u-boot.elf
}
```

3.3 Bootgen을 사용한 **BOOT.BIN** 이미지 생성

petalinux-package 명령을 사용하여 BIF 파일을 명시적으로 지정하지 않고 구성 요소로부터 BOOT.BIN을 생성할 수 있습니다.

1. **Golden** 이미지 (**BOOT_A.bin**) 생성:

Bash

```
petalinux-package --boot --fsbl images/linux/zynqmp_fsbl.elf \
  --pmufw images/linux/pmufw.elf \
  --atf images/linux/bl31.elf \
  --u-boot images/linux/u-boot.elf \
  --force -o images/linux/BOOT_A.bin
```

2. **Update** 이미지 (**BOOT_B.bin**) 생성:

Bash

동일한 구성 요소로 두 번째 이미지 생성

```
petalinux-package --boot --fsbl images/linux/zynqmp_fsbl.elf \
  --pmufw images/linux/pmufw.elf \
  --atf images/linux/bl31.elf \
  --u-boot images/linux/u-boot.elf \
  --force -o images/linux/BOOT_B.bin
```

3.4 QSPI 플래시 프로그래밍

생성된 두 개의 BOOT.BIN을 표 1의 boot_a 및 boot_b 파티션 오프셋에 프로그래밍합니다. Vitis, JTAG/U-Boot sf 명령 또는 Linux flashcp를 사용할 수 있습니다.

- 예: Linux에서 flashcp 사용 (SD 카드로 부팅 후):

Bash

mtd utils가 설치되어 있어야 함

flash_erase /dev/mtd0 0 0 # boot_a 파티션 전체 지우기

flashcp images/linux/BOOT_A.bin /dev/mtd0

flash_erase /dev/mtd1 0 0 # boot_b 파티션 전체 지우기

flashcp images/linux/BOOT_B.bin /dev/mtd1

3.5 U-Boot에서 Tier 1 페일오버 수동 트리거 (PoC)

CSU_MULTI_BOOT 레지스터 주소는 0xFFCA0010입니다.⁷ BOOT_B.bin 오프셋(0x2000000)에 해당하는 MultiBoot 값은 $0x2000000 / 0x8000 = 0x400$ (1024)입니다.⁶

- BOOT_A (0x0)로 부팅 (기본값):

코드 스니펫

ZynqMP> mw 0xFFCA0010 0x0

ZynqMP> reset

- BOOT_B (0x2000000)로 강제 부팅:

코드 스니펫

ZynqMP> mw 0xFFCA0010 0x400

ZynqMP> reset

이 명령은 1계층(Tier 1) 하드웨어 페일오버 메커니즘이 올바르게 작동하는지 검증합니다.

섹션 4: U-Boot A/B 부트 로직 구현 (Tier 2 제어)

(Implementing U-Boot A/B Boot Logic - Tier 2 Control)

BOOT.BIN이 성공적으로 로드되면 U-Boot가 실행됩니다. 이제 U-Boot가 Kernel/RootFS의 A/B 슬롯을 선택하고 부팅 실패를 처리하는 2계층(Tier 2) 로직을 구현합니다.

4.1 U-Boot 환경 설정 (petalinux-config -c u-boot)

A/B 로직을 위한 핵심 U-Boot 기능을 활성화합니다. petalinux-config -c u-boot 명령으로 U-Boot의 menuconfig를 엽니다.

- 부트 카운팅 활성화:⁹
 - Boot options --->
 - [*] Enable bootcount limit support (CONFIG_BOOTCOUNT_LIMIT=y)
- U-Boot 환경 이중화 활성화:
 - Environment --->
 - [*] Support for redundant environment (CONFIG_ENV_OFFSET_REDUND=y)
- 환경 변수 위치 설정 (표 1 기준):
 - Environment --->
 - (0x4000000) Environment Offset (bootenv 파티션 시작 주소)
 - (0x4040000) Redundant environment offset (bootenv_redund 시작 주소)
 - (0x40000) Environment Size (256K)
- boot.scr 스크립트 활성화:
 - Boot options ---> [*] Enable boot.scr script

4.2 PetaLinux RootFS에 u-boot-tools 추가

Linux 유저스페이스에서 U-Boot 환경 변수를 수정(fw_setenv)하기 위해 u-boot-tools 패키지가 필요합니다.²²

- petalinux-config -c rootfs
- Filesystem Packages ---> misc ---> u-boot-tools --->
 - [*] u-boot-tools
 - [*] u-boot-tools-fw-utils⁷⁵
- (PetaLinux 2022.1 이상에서는 libubootenv가 u-boot-tools-fw-utils를 대체할 수 있으며, petalinux-config -c rootfs -> user packages에서 libubootenv와 libubootenv-bin을 추가합니다.)⁷⁶

4.3 핵심 boot.scr A/B 페일오버 스크립트 작성 (PoC)

이 스크립트는 Android/RAUC/Mender에서 사용하는 표준 A/B 부팅 로직을 모방합니다.¹¹
`bootcount`⁹는 전체 부팅 시도를 계산하고, `boot_x_left`¹¹는 슬롯별 시도를 계산합니다.

본 PoC에서는 `boot_x_left` 방식이 A/B 슬롯 관리에 더 직관적이므로 주 로직으로 사용합니다.
`bootcount`는 Linux 시스템 행(hang)을 감지하는 WDT 리셋을 계산하는 데 보조적으로 사용합니다.

통합 로직:

1. WDT 리셋으로 부팅 시 U-Boot는 `bootcount`를 증가시킵니다.⁹
2. `boot.scr`는 `bootcount`가 0보다 큰지 확인합니다.
3. 만약 0보다 크면(WDT 리셋 발생), 현재 부팅 중인 슬롯의 `boot_x_left` 카운터를 강제로 1 감소시킵니다.
4. `boot_x_left`가 0이 되면, 해당 슬롯은 "bad"로 간주하고 다른 슬롯으로 패일오버합니다.
5. Linux가 성공적으로 부팅되면, `boot-success` 서비스가 두 카운터 모두 리셋합니다 (`fw_setenv bootcount 0; fw_setenv boot_a_left 3`).

boot.cmd (boot.scr의 소스 파일) 예제 ¹⁴:

코드 스니펫

```
echo "## Starting A/B Boot Script..."
```

```
# 1. Initialize default state if variables don't exist
```

```
test -n "${boot_order}" |
```

```
| setenv boot_order "A B"
```

```
test -n "${boot_a_left}" |
```

```
| setenv boot_a_left 3
```

```
test -n "${boot_b_left}" |
```

```
| setenv boot_b_left 3
```

```
test -n "${bootlimit}" |
```

```
| setenv bootlimit 3 # Max 3 boot failures before Tier 1 fallback
```

```
# 2. Check for boot failures (WDT reset or failed boot)
```

```
# bootcount is automatically incremented by U-Boot if CONFIG_BOOTCOUNT_LIMIT is set [9, 78]
```

```
if test ${bootcount} -gt 0; then
    echo "WARN: Boot failure detected (count: ${bootcount}). Decrementing slot tries."
```

```
    # Find which slot we were *trying* to boot (set by 'boot_order_first')
    setenv current_slot ${boot_order_first}
    if test "${current_slot}" = "A"; then
        setexpr boot_a_left ${boot_a_left} - 1
        echo "Slot A tries remaining: ${boot_a_left}"
    elif test "${current_slot}" = "B"; then
        setexpr boot_b_left ${boot_b_left} - 1
        echo "Slot B tries remaining: ${boot_b_left}"
    fi
```

```
    # Reset bootcount for next attempt, but check against bootlimit
    if test ${bootcount} -ge ${bootlimit}; then
        echo "FATAL: Boot failure limit reached. Triggering Tier 1 Fallback."
        # Run altbootcmd (Tier 1 Fallback)
        run altbootcmd
    else
        # Save decremented slot counter and reset bootcount
        setenv bootcount 0
        saveenv
    fi
fi
```

```
# Define altbootcmd: Tier 1 Fallback (return to Golden BOOT_A at 0x0)
setenv altbootcmd "echo Triggering Tier 1 Fallback to Golden Image...; \
    setenv boot_a_left 3; \
    setenv boot_b_left 3; \
    setenv boot_order 'A B'; \
    setenv bootcount 0; \
    saveenv; \
    mw 0xFFCA0010 0x0; \
    reset"
```

```
# 3. Iterate through boot order to find a valid slot
setenv bootargs_set 0
for slot in ${boot_order}; do
    if test ${bootargs_set} -eq 0; then
        echo "Attempting to boot slot ${slot}..."
        if test "${slot}" = "A"; then
            if test ${boot_a_left} -gt 0; then
                echo "Found valid slot A. Tries left: ${boot_a_left}"
```

```

        setenv mtd_kernel "kernel_a"
        setenv mtd_rootfs "rootfs_a"
        setenv mtd_rootfs_num 5 # From Table 1, mtd5
        setenv slot_id "A"
        setenv bootargs_set 1
    fi
    elif test "${slot}" = "B"; then
        if test ${boot_b_left} -gt 0; then
            echo "Found valid slot B. Tries left: ${boot_b_left}"
            setenv mtd_kernel "kernel_b"
            setenv mtd_rootfs "rootfs_b"
            setenv mtd_rootfs_num 7 # From Table 1, mtd7
            setenv slot_id "B"
            setenv bootargs_set 1
        fi
    fi
fi
done

```

4. If a valid slot was found, set bootargs and attempt boot

```

if test ${bootargs_set} -eq 1; then
    echo "Loading from ${mtd_kernel} and ${mtd_rootfs}..."
    # Set dynamic bootargs
    setenv bootargs "console=ttyPS0,115200 root=/dev/mtdblock${mtd_rootfs_num}
rootfstype=jffs2 rw rauc.slot=${slot_id}"

    # Store the slot we are attempting to boot
    setenv boot_order_first ${slot_id}
    saveenv

    echo "Loading Kernel from MTD (${mtd_kernel})..."
    sf probe 0 0 0
    # Load kernel from QSPI partition 'kernel_a' or 'kernel_b' to RAM
    sf read ${loadaddr} ${mtd_kernel} 0x1C00000 # ${loadaddr} is usually 0x10000000

    echo "Booting Kernel..."
    bootm ${loadaddr}

    # If bootm fails, it returns here
    echo "ERROR: Kernel bootm command failed! Triggering reset..."
    reset

```

5. If no valid slot found (A and B exhausted)


```

else
    echo "FATAL: No valid boot slots (A or B) remaining!"
    # Trigger Tier 1 fallback defined in altbootcmd
    run altbootcmd
fi

```

4.4 boot.scr 생성 및 PetaLinux 프로젝트에 통합

1. 위 내용을 boot.cmd 파일로 저장합니다.
2. mkimage (PetaLinux 툴체인에 포함)를 사용하여 boot.scr을 생성합니다.⁷⁹
 Bash

```
mkimage -A arm64 -T script -C none -n "A/B Boot Script" -d boot.cmd boot.scr
```
3. PetaLinux meta-user 레이어에 boot.scr을 복사하고 .bbappend를 생성하여 빌드에 포함시킵니다.¹⁵ A/B 시스템에서는 boot.scr이 커널/루트FS와 분리되어 U-Boot에 의해 먼저 로드되어야 합니다.
 - 디렉토리 생성: `mkdir -p project-spec/meta-user/recipes-bsp/u-boot/u-boot-zynq-scr/`
 - boot.cmd 복사: `cp boot.cmd project-spec/meta-user/recipes-bsp/u-boot/u-boot-zynq-scr/`
 - bbappend 파일 생성: `vi project-spec/meta-user/recipes-bsp/u-boot/u-boot-zynq-scr.bbappend`
 - bbappend 내용 (PetaLinux 2021.1+ 기준)²⁰:
 코드 스니펫

```

FILESEXTRAPATHS_prepend := "${THISDIR}/${PN}:"
SRC_URI_append = " file://boot.cmd"

# This bbappend forces the build to use our custom boot.cmd
do_compile_prepend() {
    cp ${WORKDIR}/boot.cmd ${S}/boot.cmd.default.initrd
}

```
 - PetaLinux는 빌드 시 이 boot.cmd를 boot.scr로 컴파일하고, petalinux-package는 이를 BOOT.BIN에 자동으로 패키징합니다.

섹션 5: Linux 유저스페이스 연동 및 업데이트 관리

(Linux Userspace Integration and Update Management)

시스템이 성공적으로 Linux로 부팅되면, "부팅 성공"을 U-Boot에 알리고(commit), 시스템 행(hang)을 감지하며, 다음 업데이트를 준비하는 로직이 필요합니다.

5.1 fw_setenv를 위한 /etc/fw_env.config 설정

u-boot-tools의 fw_printenv/fw_setenv가 U-Boot 환경 변수 MTD 파티션을 찾도록 설정해야 합니다.²¹ 이중화 환경 변수 파티션(bootenv, bootenv_redund)을 사용합니다.

- RootFS에 /etc/fw_env.config 파일을 생성합니다 (PetaLinux에서는 project-spec/meta-user/recipes-core/base-files/base-files_%.bbappend를 통해 추가).
- /etc/fw_env.config 내용 ²²:

코드 스니펫

```
# Configuration file for fw_(printenv/setenv) utility.  
# Based on Table 1 Partition Layout
```

# MTD device name	Device offset	Env. size
/dev/mtd2	0x0	0x40000
/dev/mtd3	0x0	0x40000

- 검증: Linux 부팅 후, fw_printenv boot_order를 실행하여 U-Boot 변수가 올바르게 읽히는지 확인합니다.⁷⁴ Configuration file wrong or corrupted 오류가 발생하면 ⁷⁶, 오프셋과 MTD 장치 번호를 확인해야 합니다.

5.2 하드웨어 워치독(Cadence WDT) 활성화 (시스템 행 감지)

Zynq MPSoC/RFSoc의 WDT는 PMU 펌웨어와 연동되어야만 시스템 리셋을 올바르게 트리거할 수 있습니다.²⁴

1. PMU 펌웨어 설정 (필수): ²⁴

- project-spec/meta-user/recipes-bsp/pmu-firmware/pmu-firmware_%.bbappend 파일을 생성합니다.
- 다음 내용을 추가하여 오류 관리 모듈을 활성화합니다:

코드 스니펫

```
# Enable Error Management (EM) for WDT Reset
```

```
YAML_COMPILER_FLAGS_append = " -DENABLE_EM"
```

2. 디바이스 트리 설정 (**system-user.dtsi**): ²⁴

- FPD(Full Power Domain) 위치독을 활성화하고 리셋을 트리거하도록 설정합니다.
- system-user.dtsi에 다음을 추가합니다:

```
DTS
/* Enable FPD WDT and set it to trigger system reset */
&watchdog0 {
    status = "okay";
    reset-on-timeout; /* Tell PMUFW to reset system [24] */
    timeout-sec = ; /* 60 second timeout */
};
```

3. **systemd** 위치독 연동: ⁸⁴

- systemd가 /dev/watchdog0 장치를 자동으로 'pet'(먹이 주기)하도록 설정합니다.
- /etc/systemd/system.conf 파일을 수정합니다 (PetaLinux에서는 meta-user 레이어에서 오버라이드).
- 다음 라인의 주석을 제거하고 값을 설정합니다:

```
코드 스니펫
RuntimeWatchdogSec=30s
ShutdownWatchdogSec=10min
```

- 작동 원리: systemd (PID 1)는 30초마다 /dev/watchdog0에 쓰기 작업을 수행합니다. 만약 Linux 유저스페이스(또는 커널)가 30초 이상 응답하지 않으면 ⁸⁷, systemd가 'pet'을 멈춥니다. 60초(디바이스 트리 timeout-sec)가 지나면 하드웨어 WDT가 타임아웃되고, PMU ²⁵가 이를 감지하여 시스템을 리셋시킵니다.

5.3 "부팅 성공" 확정 (**boot-success**) 서비스 생성

목적: Linux 부팅이 성공적으로 완료되었음을 U-Boot에 알려, WDT 리셋으로 인한 불필요한 페일오버를 방지합니다. 이 서비스는 boot.cmd 스크립트에서 감소시킨 boot_x_left와 bootcount를 리셋합니다.

- **boot-success.service (systemd 서비스 파일):**

- project-spec/meta-user/recipes-core/systemd/systemd-bbappend에 boot-success.service 파일 추가.

```
코드 스니펫
[Unit]
Description=Mark Boot as Successful (A/B Boot Commit)
After=multi-user.target
```

```
Type=oneshot
ExecStart=/usr/bin/boot-commit.sh
```

```
[Install]
WantedBy=multi-user.target
```

- **/usr/bin/boot-commit.sh (셸 스크립트):** ⁴⁰
 - PetaLinux의 meta-user 레이어에 이 스크립트를 추가합니다.

```
Bash
#!/bin/sh
```

```
echo "## Committing A/B Boot..."
```

```
# 1. Find out which slot we just booted from (via kernel cmdline) [14, 89]
SLOT_ID=$(cat /proc/cmdline | grep -o 'rauc.slot=[^]*' | cut -d'=' -f2)
```

```
# 2. Reset the boot attempt counter for that slot
```

```
if; then
    fw_setenv boot_a_left 3
elif; then
    fw_setenv boot_b_left 3
else
    echo "ERROR: Could not determine booted slot. No commit."
    exit 1
fi
```

```
# 3. Reset the global bootcount (WDT failure counter)
fw_setenv bootcount 0
```

```
echo "## Slot ${SLOT_ID} committed successfully."
```

5.4 현장 업데이트(OTA) PoC 워크플로우 스크립트

다음은 비활성 파티션에 업데이트를 적용하는 PoC 스크립트 예제입니다.³⁶ 실제 환경에서는 RAUC/Mender가 이 로직을 수행합니다.¹⁰

- **update-firmware.sh (예제):**

```
Bash
#!/bin/sh
```

```
NEW_KERNEL_IMG="new_kernel.ub"
NEW_ROOTFS_IMG="new_rootfs.jffs2"
```

1. Determine current and inactive slots

```
CURRENT_SLOT=$(cat /proc/cmdline | grep -o 'rauc.slot=[^']*' | cut -d'=' -f2)
```

```
if; then
```

```
    TARGET_SLOT="B"
```

```
    TARGET_KERNEL_MTD="/dev/mtd6" # mtd6 = kernel_b (Table 1)
```

```
    TARGET_ROOTFS_MTD="/dev/mtd7" # mtd7 = rootfs_b (Table 1)
```

```
elif; then
```

```
    TARGET_SLOT="A"
```

```
    TARGET_KERNEL_MTD="/dev/mtd4" # mtd4 = kernel_a
```

```
    TARGET_ROOTFS_MTD="/dev/mtd5" # mtd5 = rootfs_a
```

```
else
```

```
    echo "FATAL: Unknown boot slot. Aborting."
```

```
    exit 1
```

```
fi
```

```
echo "Updating inactive slot ${TARGET_SLOT}..."
```

2. Flash new images to inactive MTD partitions

(Assume new images are in /tmp/)

```
echo "Flashing Kernel to ${TARGET_KERNEL_MTD}..."
```

```
flash_erase ${TARGET_KERNEL_MTD} 0 0
```

```
flashcp /tmp/${NEW_KERNEL_IMG} ${TARGET_KERNEL_MTD}
```

```
echo "Flashing RootFS to ${TARGET_ROOTFS_MTD}..."
```

```
flash_erase ${TARGET_ROOTFS_MTD} 0 0
```

```
flashcp /tmp/${NEW_ROOTFS_IMG} ${TARGET_ROOTFS_MTD}
```

3. Set U-Boot env to try the new slot on next boot [40]

```
echo "Setting boot order to ${TARGET_SLOT} ${CURRENT_SLOT}..."
```

```
fw_setenv boot_order "${TARGET_SLOT} ${CURRENT_SLOT}"
```

4. Set attempt counts for the new slot

```
if; then
```

```
    fw_setenv boot_a_left 3
```

```
else
```

```
    fw_setenv boot_b_left 3
```

```
fi
```

```
echo "Update complete. Rebooting to slot ${TARGET_SLOT}."
```

```
reboot
```

섹션 6: PoC 검증 및 장애 시나리오 테스트

(PoC Verification and Failure Scenario Testing)

설계된 아키텍처가 의도한 대로 작동하는지 검증하기 위해 고의로 장애를 주입하여 페일오버 시나리오를 테스트합니다.

표 2: A/B 페일오버 테스트 매트릭스

- 배경: 이 테이블은 PoC의 핵심 기능(장애 감지 및 페일오버)을 검증하기 위한 구체적인 테스트 케이스를 정의합니다. 각 테스트는 1계층(BootROM) 또는 2계층(U-Boot/Kernel)의 장애 시나리오를 시뮬레이션합니다.³⁸ 각 테스트는 (1) 장애 주입 방법⁹¹, (2) 예상되는 페일오버 메커니즘, (3) UART 콘솔 로그를 통한 확인 방법을 명시합니다.

테스트 케이스	장애 계층	장애 주입 방법 (명령어)	예상 결과 (페일오버 메커니즘)	확인 방법 (UART 로그)
Test 1: Linux 시스템 행	2계층 (Kernel/Runtime e)	부팅 성공 후 (슬롯 A), systemd 워치독 서비스 중지: systemctl stop systemd-watc hdog.service 또는 커널 패닉 유도: echo c > /proc/sysrq-tri gger ³⁷	1. systemd가 WDT 'pet' 중지. 2. 60초 후 하드웨어 WDT 타임아웃, PMU가 시스템 리셋. ²⁴ 3. U-Boot가 bootcount > 0 감지. ⁹ 4. boot.scr0l boot_a_left 1 감소 후 saveenv. 5. 3회 반복 시 boot_a_left가	"WARN: Boot failure detected" 메시지. "Slot A tries remaining: 2" ... "Found valid slot B."

			0이 됨. 6. boot.scr0이 boot_order에 따라 슬롯 B 부팅 시도. ¹⁴	
Test 2: 커널 부팅 실패	2계층 (U-Boot/Kernel Load)	슬롯 A의 kernel_a 파티션을 고의로 손상: flash_erase /dev/mtd4 0 1 (헤더만 지움) ⁹²	1. U-Boot가 슬롯 A 부팅 시도. 2. sf read 또는 bootm 명령 실패. 3. boot.scr의 bootm 이후 로직(reset) 실행. 4. U-Boot가 bootcount > 0 감지. 5. boot.scr0이 boot_a_left 감소. 6. 3회 반복 후 슬롯 B로 파일오버.	"ERROR: Kernel bootm command failed!" "WARN: Boot failure detected" "Found valid slot B."
Test 3: BOOT.BIN 손상	1계층 (BootROM/FSBL)	슬롯 A의 boot_a 파티션 헤더를 손상: flash_erase /dev/mtd0 0 1 ³⁸	1. BootROM이 0x0에서 부트 헤더 검증 시도. ⁸ 2. 헤더(시그니처) 검증 실패. 3. BootROM이	FSBL 부트 배너가 평소와 다르게 늦게 나타나거나, FSBL 로그에 "MultiBootOffset: 0x400" ⁹³ 이 표시됨 (BootROM이 CSU_MULTI_B

			32KB 단위로 다음 이미지 검색. ⁶ 4. 0x20000000 (boot_b)에서 유효한 헤더 발견. 5. BootROM이 BOOT_B.bin을 로드.	OOT 레지스터를 업데이트함).
--	--	--	---	-------------------------

섹션 7: 결론 및 전문가 제언: PoC에서
양산(Production)으로

(Conclusion and Expert Recommendations: From PoC to Production)

7.1 PoC 아키텍처 요약 및 평가

본 보고서에서 제시한 2계층(Two-Tiered) 이중화 아키텍처는 Zynq UltraScale+ RFSoc DFE
보드에서 FSBL부터 Linux 커널까지의 전체 부트 체인에 대한 강력한 장애 복구 메커니즘을
제공합니다.

- **1계층(BootROM MultiBoot)**은 BOOT.BIN의 치명적 손상을 방지하며,
- **2계층(U-Boot 스크립트)**은 커널/RootFS 업데이트 및 런타임 장애를 관리합니다.

하드웨어 WDT와 U-Boot bootcount, boot_x_left 변수의 연동은 런타임 장애가 부팅 슬롯
페일오버로 이어지는 '신뢰 체인'을 완성합니다.

7.2 PoC에서 양산으로: 검증된 OTA 프레임워크로의 전환

본 PoC에서 구축한 아키텍처는 강력한 기반을 제공하지만, 프로덕션 환경에서는 한계가 명확합니다. PoC의 업데이트 스크립트(update-firmware.sh)는 업데이트 중 전원 차단에 매우 취약합니다.⁴⁷ flashcp 도중 전원이 나가면 비활성 슬롯이 손상됩니다. 또한 업데이트 패키지(이미지)의 무결성, 인증, 암호화⁴⁷를 보장하지 않습니다.

Xilinx(AMD)는 공식 OTA 레퍼런스 디자인을 제공하지 않지만, 임베디드 리눅스 커뮤니티에서는 RAUC, Mender, SWUpdate와 같은 오픈소스 프레임워크를 강력히 권장합니다.⁹⁵

이러한 프레임워크는 본 PoC에서 구축한 기반 위에서 작동합니다.

- **RAUC (Robust Auto-Update Controller):**¹⁰ '번들(bundle)'⁹⁷ 기반의 원자적(atomic) 업데이트를 제공하며, meta-rauc Yocto 레이어를 통해 PetaLinux에 쉽게 통합됩니다. RAUC는 본 PoC에서 구현한 것과 동일한 U-Boot 변수(BOOT_ORDER, BOOT_A_LEFT, BOOT_B_LEFT)를 사용하여 부트로더와 통신합니다.¹¹
- **Mender:**¹² 중앙 서버 기반의 OTA 관리에 중점을 둔 포괄적인 프레임워크입니다. meta-mender-zynq⁹⁹ 레이어를 제공하며, mender commit¹⁰⁰ 명령을 통해 부팅 성공을 확정합니다.

최종 결론: 본 PoC는 Zynq MPSoC/RFSoc의 MultiBoot 및 U-Boot 환경 변수라는 저수준 하드웨어/부트로더 제어를 마스터하기 위한 필수 과정입니다. 이 기반이 확립되면, RAUC 또는 Mender와 같은 고수준 프레임워크를 도입하여 PoC의 수동 스크립트를 대체하고, 서명된 번들, D-Bus API, 강력한 상태 관리를 갖춘 프로덕션급 OTA 솔루션을 구현해야 합니다.

7.3 RFSoc DFE 시스템을 위한 최종 권장 사항

RFSoc DFE¹와 같이 미션 크리티컬한 통신 시스템에서는 1계층(하드웨어 MultiBoot)과 2계층(소프트웨어 A/B)을 결합한 이중화 전략이 필수입니다.

1. **Golden Image 보호:** 1계층 BOOT_A (Golden) 이미지는 0x0에 두고, 가능하면 QSPI 플래시의 쓰기 방지(Write-Protect) 기능을 활성화하여 현장에서 절대 수정할 수 없도록 보호하는 것을 강력히 권장합니다.
2. **Ultimate Failsafe 구현:** 모든 업데이트는 1계층 BOOT_B 및 2계층 System_B 파티션을 대상으로 수행합니다. BOOT_B의 FSBL/U-Boot가 실패할 경우, 시스템 WDT가 이를 감지하고²⁵, U-Boot altbootcmd가 CSU_MULTI_BOOT 레지스터를 0x0으로 설정⁴²하여 BOOT_A로 복구하는 '궁극의 페일세이프(Ultimate Failsafe)' 로직을 구현해야 합니다.
3. **데이터 무결성:** RootFS를 A/B로 분리하는 것 외에도, 사용자 데이터와 설정 파일은 별도의 user_data 파티션에 저장하여(표 1 참조) 어떤 슬롯으로 부팅하든 동일한 데이터에 접근하고, 업데이트 시 사용자 설정이 초기화되는 것을 방지해야 합니다.⁴⁷

참고 자료

1. ZYNQ RFSOC DFE BACKGROUNDER - AMD, 11월 11, 2025에 액세스, <https://www.xilinx.com/publications/backgrounders/zynq-rfsoc-dfe-backgrounder.pdf>
2. Zynq UltraScale+ RFSoc Data Sheet: Overview (DS889) - AMD Technical Information Portal, 11월 11, 2025에 액세스, <https://docs.amd.com/api/khub/documents/Pa6T7NxoJJAPnqu~CbhDfg/content>
3. Zynq UltraScale+ RFSoc DFE Product Data Sheet: Overview (DS883), 11월 11, 2025에 액세스, https://docs.amd.com/api/khub/documents/l8cVVSkkDRNOBquT_p_Auw/content
4. RFSoc Device Configuration - UG1271 - AMD Technical Information Portal, 11월 11, 2025에 액세스, <https://docs.amd.com/r/en-US/ug1271-zcu111-eval-bd/RFSoc-Device-Configuration>
5. Boot Process Overview - 2025.1 English - UG1137, 11월 11, 2025에 액세스, <https://docs.amd.com/r/en-US/ug1137-zynq-ultrascale-mpsoc-swdev/Boot-Process-Overview>
6. U-Boot Linux Software Updates on Zynq UltraScale+ MPSoC ..., 11월 11, 2025에 액세스, <https://docs.foundries.io/92/reference-manual/security/boot-software-updates-zynqmp.html>
7. Multi-boot for ZYNQMP - Adaptive Support - AMD, 11월 11, 2025에 액세스, https://adaptivesupport.amd.com/s/question/0D54U00008HaHhLSAV/multiboot-for-zynqmp?language=en_US
8. Fallback and MultiBoot Flow - 2025.1 English - UG1137, 11월 11, 2025에 액세스, <https://docs.amd.com/r/en-US/ug1137-zynq-ultrascale-mpsoc-swdev/Fallback-and-MultiBoot-Flow>
9. Boot Count Limit — Das U-Boot unknown version documentation, 11월 11, 2025에 액세스, <https://docs.u-boot.org/en/latest/api/bootcount.html>
10. Bringing RAUC A/B Updates to More Linux Devices - TIB AV-Portal, 11월 11, 2025에 액세스, <https://av.tib.eu/media/56847>
11. RAUC Bootmeth — Das U-Boot unknown version documentation, 11월 11, 2025에 액세스, <https://docs.u-boot.org/en/latest/develop/bootstd/rauc.html>
12. mendersoftware/mender: Mender over-the-air software updater client. - GitHub, 11월 11, 2025에 액세스, <https://github.com/mendersoftware/mender>
13. How Mender over-the-air (OTA) updates work, 11월 11, 2025에 액세스, <https://mender.io/engineers/how-mender-works>
14. Solved: Change U-Boot Variables for Different rootfs / boo ..., 11월 11, 2025에 액세스, <https://community.st.com/t5/stm32-mpus-embedded-software-and/change-u-boot-variables-for-different-rootfs-bootfs-partitions/td-p/206170>
15. PetaLinux Tools Documentation Reference Guide, 11월 11, 2025에 액세스, https://www.xilinx.com/support/documents/sw_manuals/xilinx2022_2/ug1144-petalinux-tools-reference-guide.pdf

16. Using SD for Boot - Xilinx Wiki - Confluence, 11월 11, 2025에 액세스,
<https://xilinx-wiki.atlassian.net/wiki/spaces/A/pages/861569243>
17. QSPI Partitioning and Device Tree - Adaptive Support - AMD, 11월 11, 2025에 액세스,
https://adaptivesupport.amd.com/s/question/0D52E00006hpK0SSAU/qspi-partitioning-and-device-tree?language=en_US
18. System Example Using the Vitis IDE Create Boot Image Wizard - 2025.1 English - UG1209, 11월 11, 2025에 액세스,
<https://docs.amd.com/r/en-US/ug1209-embedded-design-tutorial/System-Example-Using-the-Vitis-IDE-Create-Boot-Image-Wizard>
19. Boot Image Format (BIF) - 2025.1 English - UG1283, 11월 11, 2025에 액세스,
<https://docs.amd.com/r/en-US/ug1283-bootgen-user-guide/Boot-Image-Format-BIF>
20. Patching U-Boot boot script in Petalinux 2021.1 - Tux Engineering, Inc., 11월 11, 2025에 액세스,
<https://tuxengineering.com/blog/2021/09/22/Patching-U-Boot-boot-script-in-Petalinux-2021.1.html>
21. U-boot env vars - Technical Support - Toradex Community, 11월 11, 2025에 액세스, <https://community.toradex.com/t/u-boot-env-vars/26695>
22. How to modify U-boot configuration in Linux user space - NXP Community, 11월 11, 2025에 액세스,
<https://community.nxp.com/t5/Layerscape/How-to-modify-U-boot-configuration-in-Linux-user-space/td-p/918642>
23. Enabling Watchdog Timers on the Zynq Ultrascale+ MPSoC: Step by Step, 11월 11, 2025에 액세스,
<https://cornersoftsolutions.com/enabling-watchdog-timers-on-the-zynq-ultrascale-mpsoc-step-by-step/>
24. Confluence - Xilinx Wiki, 11월 11, 2025에 액세스,
<https://xilinx-wiki.atlassian.net/wiki/spaces/A/pages/18842055/Cadence+WDT+Driver?view=blog>
25. Zynq UltraScale+ MPSoC Software Developer Guide - AMD, 11월 11, 2025에 액세스,
https://www.xilinx.com/support/documents/sw_manuals/xilinx2022_2/ug1137-zynq-ultrascale-mpsoc-swdev.pdf
26. Zynq Booting & PetaLinux Tutorial + Demo - CERN Indico, 11월 11, 2025에 액세스,
<https://indico.cern.ch/event/799275/contributions/3413736/attachments/1861349/3059284/PetaLinux.pdf>
27. Zynq UltraScale+ RFSoc - Xilinx Wiki, 11월 11, 2025에 액세스,
<https://xilinx-wiki.atlassian.net/wiki/spaces/A/pages/189530203/Zynq%2BUltraScale%2BRFSoc>
28. Zynq UltraScale+ FSBL - Xilinx Wiki - Confluence, 11월 11, 2025에 액세스,
<https://xilinx-wiki.atlassian.net/wiki/spaces/A/pages/18842019/Zynq+UltraScale+FSBL>
29. PMU Firmware > Using FSBL to load PMUFW - Xilinx Wiki - Confluence, 11월 11, 2025에 액세스,

- <https://xilinx-wiki.atlassian.net/wiki/spaces/A/pages/18841724/PMU+Firmware>
30. Boot and Configuration — Embedded Design Tutorials 2022.2 documentation - GitHub Pages, 11월 11, 2025에 액세스,
<https://xilinx.github.io/Embedded-Design-Tutorials/docs/2022.2/build/html/docs/Introduction/ZynqMPSoC-EDT/8-boot-and-configuration.html>
 31. Building the ZynqMP PMU firmware, the simple way - Luca Ceresoli, 11월 11, 2025에 액세스, <https://lucaceresoli.net/zynqmp-pmufw-builder/>
 32. PMU Firmware Loading Options - 2025.1 English - UG1137, 11월 11, 2025에 액세스, <https://docs.amd.com/r/en-US/ug1137-zynq-ultrascale-mpsoc-swdev/PMU-Firmware-Loading-Options>
 33. Creating boot.scr for u-boot - Stack Overflow, 11월 11, 2025에 액세스, <https://stackoverflow.com/questions/29342821/creating-boot-scr-for-u-boot>
 34. Customize U-boot script to modify u-boot process - Stack Overflow, 11월 11, 2025에 액세스, <https://stackoverflow.com/questions/70823798/customize-u-boot-script-to-modify-u-boot-process>
 35. Implementing A/B System Updates with U-Boot - Michael Opdenacker, Bootlin - YouTube, 11월 11, 2025에 액세스, <https://www.youtube.com/watch?v=TldwB9p1KeQ>
 36. Implementing A/B System Updates with U-Boot - Bootlin, 11월 11, 2025에 액세스, <https://bootlin.com/pub/conferences/2022/elce/opdenacker-implementing-A-B-system-updates-with-u-boot/opdenacker-implementing-A-B-system-updates-with-u-boot.pdf>
 37. FPD_SWDT not working if Linux boot hangs when PL not configured - Adaptive Support, 11월 11, 2025에 액세스, https://adaptivesupport.amd.com/s/question/0D52E00006hplWnSAE/fpdswdt-not-working-if-linux-boot-hangs-when-pl-not-configured?language=en_US
 38. Zynq-7000 Fallback to Golden Image (multiboot) - Adaptive Support - AMD, 11월 11, 2025에 액세스, https://adaptivesupport.amd.com/s/question/0D52E00006hpfHhSAI/zynq7000-fallback-to-golden-image-multiboot?language=en_US
 39. Zynq 7000 - Use multiboot to set recovery image - Adaptive Support - AMD, 11월 11, 2025에 액세스, https://adaptivesupport.amd.com/s/question/0D54U00006sYn2vSAC/zynq-7000-use-multiboot-to-set-recovery-image?language=en_US
 40. A/B firmware update with swupdate handler: end up in update loop · Issue #460 - GitHub, 11월 11, 2025에 액세스, <https://github.com/Azure/iot-hub-device-update/issues/460>
 41. Compilation of u-boot with CONFIG_BOOTCOUNT_LIMIT - Toradex Community, 11월 11, 2025에 액세스, <https://community.toradex.com/t/compilation-of-u-boot-with-config-bootcount-limit/5807>
 42. FSBL Image Fallback and Multiboot - UG585 - AMD Technical Information Portal, 11월 11, 2025에 액세스, <https://docs.amd.com/r/en-US/ug585-zynq-7000-SoC-TRM/FSBL-Image-Fallback>

-and-Multiboot

43. Configuring and Building Linux Using PetaLinux - 2025.1 English - UG1209, 11월 11, 2025에 액세스,
<https://docs.amd.com/r/en-US/ug1209-embedded-design-tutorial/Configuring-and-Building-Linux-Using-PetaLinux>
44. Adam Taylor's MicroZed Chronicles, Part 219, UltraZed Edition: How to Build PetaLinux for the Zynq UltraScale+ MPSoC - Adaptive Support, 11월 11, 2025에 액세스, <https://adaptivesupport.amd.com/s/article/799417>
45. PetaLinux Building and System Customization — Vitis™ Tutorials 2022.1 documentation, 11월 11, 2025에 액세스,
https://xilinx.github.io/Vitis-Tutorials/2022-1/build/html/docs/Vitis_Platform_Creation/Feature_Tutorials/02_petalinux_customization/README.html
46. Boot and Configuration — Embedded Design Tutorials 2021.1 documentation - GitHub Pages, 11월 11, 2025에 액세스,
<https://xilinx.github.io/Embedded-Design-Tutorials/docs/2021.1/build/html/docs/Introduction/ZynqMPSoC-EDT/8-boot-and-configuration.html>
47. Robust OTA updates with A/B Partitions for Linux devices - Mender, 11월 11, 2025에 액세스,
<https://mender.io/blog/robust-ota-updates-with-partitions-for-linux-devices>
48. Integration — RAUC 1.14.315-450e documentation, 11월 11, 2025에 액세스,
<https://rauc.readthedocs.io/en/latest/integration.html>
49. How to Modify U-Boot Environment Variables in PetaLinux - FPGA Developer, 11월 11, 2025에 액세스,
<https://www.fpgadeveloper.com/how-to-modify-u-boot-environment-variables-in-petalinux/>
50. Accessing U-Boot environment from userspace - Beaglebone Black - General Discussion, 11월 11, 2025에 액세스,
<https://forum.beagleboard.org/t/accessing-u-boot-environment-from-userspace-beaglebone-black/32775>
51. Stm32mp15 bootcount environment variable being over... - STMicroelectronics Community, 11월 11, 2025에 액세스,
<https://community.st.com/t5/stm32-mpus-embedded-software-and/stm32mp15-bootcount-environment-variable-being-overwritten-in/td-p/645290>
52. Configuring Primary Flash Partition - 2025.1 English - UG1144, 11월 11, 2025에 액세스,
<https://docs.amd.com/r/en-US/ug1144-petalinux-tools-reference-guide/Configuring-Primary-Flash-Partition>
53. Example 10: Booting Linux from QSPI Flash - 2025.1 English - UG1165, 11월 11, 2025에 액세스,
<https://docs.amd.com/r/en-US/ug1165-zynq-embedded-design-tutorial/Example-10-Booting-Linux-from-QSPI-Flash>
54. Creating Linux Images Using PetaLinux for QSPI Flash - 2025.1 English - UG1209, 11월 11, 2025에 액세스,
<https://docs.amd.com/r/en-US/ug1209-embedded-design-tutorial/Creating-Linux-Images-Using-PetaLinux-for-QSPI-Flash>

55. README.md · master · Xilinx / Development Tools / Petalinux / Project Configuration / Booting From QSPI - GitLab, 11월 11, 2025에 액세스,
<https://xterra2.avnet.com/xilinx/zcu102/booting-from-qspi/-/blob/master/README.md>
56. Unexpected MTD partitions after QSPI-booting Petalinux - Adaptive Support - AMD, 11월 11, 2025에 액세스,
https://adaptivesupport.amd.com/s/question/0D52E00006LwsmSAC/unexpected-mtd-partitions-after-qspi-booting-petalinux?language=en_US
57. Linux ZynqMP GQSPI Driver - Xilinx Wiki, 11월 11, 2025에 액세스,
<https://xilinx-wiki.atlassian.net/wiki/display/A/Linux+ZynqMP+GQSPI+Driver>
58. Linux QSPI Boot, Partitions, and Reboot, Oh My! - element14 Community, 11월 11, 2025에 액세스,
<https://community.element14.com/technologies/fpga-group/b/blog/posts/linux-qspi-boot-partitions-and-reboot-oh-my>
59. Zybo Z7 with Petalinux, unable to create partitions in QSPI flash (using SD card to boot), 11월 11, 2025에 액세스,
<https://forum.digilent.com/topic/26885-zybo-z7-with-petalinux-unable-to-create-partitions-in-qspi-flash-using-sd-card-to-boot/>
60. Configuring Device Tree - 2025.1 English - UG1144 - AMD Technical Information Portal, 11월 11, 2025에 액세스,
<https://docs.amd.com/r/en-US/ug1144-petalinux-tools-reference-guide/Configuring-Device-Tree>
61. Linux Driver - Digilent Forum, 11월 11, 2025에 액세스,
<https://forum.digilent.com/topic/9174-linux-driver/>
62. 5. Create boot files for ultrascale — OmpSs-2@FPGA User Guide, 11월 11, 2025에 액세스,
<https://pm.bsc.es/ftp/ompss-2-at-fpga/doc/user-guide-3.1.1/build-ultrascale-boot.html>
63. Getting Started With PetaLinux : 8 Steps - Instructables, 11월 11, 2025에 액세스,
<https://www.instructables.com/Getting-Started-With-PetaLinux/>
64. Configuration and Booting | High Level Synthesis Design Flow, 11월 11, 2025에 액세스,
https://xilinx.github.io/xup_embedded_system_design_flow/lab8.html
65. Bootgen User Guide, 11월 11, 2025에 액세스,
https://www.xilinx.com/support/documents/sw_manuals/xilinx2022_2/ug1283-bootgen-user-guide.pdf
66. Boot Image Format (BIF) - 2022.1 English - UG1400, 11월 11, 2025에 액세스,
<https://docs.amd.com/r/2022.1-English/ug1400-vitis-embedded/Boot-Image-Format-BIF>
67. offset - 2025.1 English - UG1283, 11월 11, 2025에 액세스,
<https://docs.amd.com/r/en-US/ug1283-bootgen-user-guide/offset>
68. Simplest of Simple Zynq Multiboot - How? - Adaptive Support - AMD, 11월 11, 2025에 액세스,
https://adaptivesupport.amd.com/s/question/0D52E00006iHv4xSAC/simplest-of-simple-zynq-multiboot-how?language=en_US
69. Boot Image Setup in Vitis - 2025.1 English - UG1209, 11월 11, 2025에 액세스,

- <https://docs.amd.com/r/en-US/ug1209-embedded-design-tutorial/Boot-Image-Setup-in-Vitis>
70. Linux/BEAGLEBK: Enable u-boot bootcount - Processors forum - TI E2E, 11월 11, 2025에 액세스,
<https://e2e.ti.com/support/processors-group/processors/f/processors-forum/769365/linux-beaglebk-enable-u-boot-bootcount>
 71. [INCOMPLETE] Xilinx Zynq 7000 - Yocto Project - Mender Hub, 11월 11, 2025에 액세스, <https://hub.mender.io/t/incomplete-xilinx-zynq-7000/669>
 72. Setting up fw_printenv to modify u-boot environment variables - RidgeRun Developer Wiki, 11월 11, 2025에 액세스,
https://developer.ridgerun.com/wiki/index.php/Setting_up_fw_printenv_to_modify_u-boot_environment_variables
 73. How to include U-Boot's "fw_printenv" tool in Petalinux 2016.4 rootfs, 11월 11, 2025에 액세스,
https://adaptivesupport.amd.com/s/question/0D52E00006hpdFmSAI/how-to-include-uboots-fwprintenv-tool-in-petalinux-20164-rootfs?language=en_US
 74. SWUpdate-Petalinux integration - Adaptive Support - AMD, 11월 11, 2025에 액세스,
https://adaptivesupport.amd.com/s/question/0D52E000079s2uOSAQ/swupdate-petalinux-integration?language=en_US
 75. Demystifying A/B Partitions: A Look at Android's Seamless Updates | Beebom, 11월 11, 2025에 액세스,
<https://beebom.com/what-are-seamless-updates-android/>
 76. U-boot boot script | Welcome to the Mike's homepage!, 11월 11, 2025에 액세스,
<https://krinkinmu.github.io/2023/11/19/u-boot-boot-script.html>
 77. Changing boot.scr for image.ub Offset Address and Size - 2025.1 English - UG1165, 11월 11, 2025에 액세스,
<https://docs.amd.com/r/en-US/ug1165-zynq-embedded-design-tutorial/Changing-boot.scr-for-image.ub-Offset-Address-and-Size>
 78. [RAUC] RAUC fails to mark slot as good --> fw_utils access error on mmc, 11월 11, 2025에 액세스,
<https://lore.rauc.io/rauc/413f3f61-684b-3ddd-cbe8-d05e78ed8753@pengutronix.de/T/>
 79. Watchdog Management - 2025.1 English - UG1137, 11월 11, 2025에 액세스,
<https://docs.amd.com/r/en-US/ug1137-zynq-ultrascale-mpsoc-swdev/Watchdog-Management>
 80. This page provides details about restart solution for Xilinx Zynq UltraScale+ SoC. - Xilinx Wiki, 11월 11, 2025에 액세스,
<https://xilinx-wiki.atlassian.net/wiki/spaces/A/pages/18841820/Zynq+UltraScale+Plus+Restart+solution>
 81. linux watchdog and systemd watchdog - Unix & Linux Stack Exchange, 11월 11, 2025에 액세스,
<https://unix.stackexchange.com/questions/125080/linux-watchdog-and-systemd-watchdog>
 82. Using Systemd for Automated System Recovery | Dell Turkmenistan, 11월 11,

- 2025에 액세스,
<https://www.dell.com/support/kbdoc/en-tm/000146106/using-systemd-for-automated-system-recovery>
83. Watchdog (Linux) - Toradex Developer Center, 11월 11, 2025에 액세스,
<https://developer.toradex.com/software/linux-resources/linux-features/watchdog-linux/>
84. watchdog: Still fed when rootfs is non-responsive · Issue #21083 - GitHub, 11월 11, 2025에 액세스, <https://github.com/systemd/systemd/issues/21083>
85. Versal Evaluation Board - System Controller - Update 7 - Xilinx Wiki - Confluence, 11월 11, 2025에 액세스,
<https://xilinx-wiki.atlassian.net/wiki/spaces/A/pages/2273738753/VersalEvaluationBoard-SystemController-Update6>
86. Reliable booting and upgrades for Zynq MPSoC - CERN Indico, 11월 11, 2025에 액세스,
<https://indico.cern.ch/event/996093/contributions/4376624/attachments/2262053/3839752/10-06-2021%20RelBoot%20and%20RelUpgrades%20for%20Zynq%20US+%20MPSoC.pdf>
87. What is a good way to test watchdog (script or command to deliberately overload system), 11월 11, 2025에 액세스,
<https://unix.stackexchange.com/questions/714910/what-is-a-good-way-to-test-watchdog-script-or-command-to-deliberately-overload>
88. MicroZed Chronicles: FPGA MultiBoot | by Adam Taylor | Medium, 11월 11, 2025에 액세스,
<https://medium.com/@aptaylorceng/microzed-chronicles-fpga-multiboot-1c0a327d5940>
89. Boot Firmware Overview - GitHub Pages, 11월 11, 2025에 액세스,
<https://xilinx.github.io/kria-apps-docs/bootfw.html>
90. 4. Fallback Procedure for A53 first then R5 in Non Secure SD boot mode - Xilinx Wiki - Confluence, 11월 11, 2025에 액세스,
<https://xilinx-wiki.atlassian.net/wiki/spaces/A/pages/18842403/Zynq+Ultrascale+MPSoC+Multiboot+and+Fallback?f=print>
91. Reference Design for Zynq Ultrascale+ OTA Update - Adaptive Support - AMD, 11월 11, 2025에 액세스,
https://adaptivesupport.amd.com/s/question/0D52E00006hpMLfSAM/reference-design-for-zynq-ultrascale-ota-update?language=en_US
92. RAUC: Safe and Secure OTA Updates for Embedded Linux, 11월 11, 2025에 액세스,
<https://rauc.io/>
93. Using RAUC — RAUC 1.14.310-e0d7 documentation, 11월 11, 2025에 액세스,
<https://rauc.readthedocs.io/en/latest/using.html>
94. Over-the-Air Updates for Embedded Linux with Mender - Magaluk Dev, 11월 11, 2025에 액세스,
<https://magaluk.dev/blog/2024-09-19-over-the-air-updates-mender/>
95. reachBAM/meta-mender-zynq: OpenEmbedded/Yocto ... - GitHub, 11월 11, 2025에 액세스, <https://github.com/reachBAM/meta-mender-zynq>
96. Upgrading | Mender documentation, 11월 11, 2025에 액세스,

- <https://docs.mender.io/client-installation/install-with-debian-package/upgrading>
97. Mender.io - Software updates for embedded Linux devices - RidgeRun Developer Wiki, 11월 11, 2025에 액세스,
https://developer.ridgerun.com/wiki/index.php/Mender.io_-_Software_updates_for_embedded_Linux_devices
98. Auto-commit for standalone mender updates - General Discussions, 11월 11, 2025에 액세스,
<https://hub.mender.io/t/auto-commit-for-standalone-mender-updates/2791>